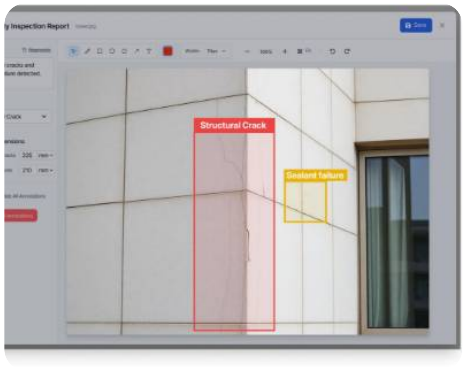
YOLO Property Inspection Integration

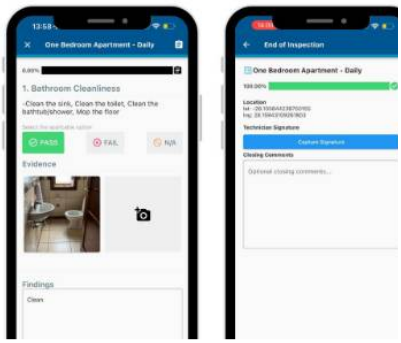
how to we can use yolo for property inspection out application iw builtwith .net core 6 apis and react as frontend and mssql

Yes. **YOLO is a very good fit for your property inspection application**, especially because you already have:

- **ASP.NET Core 6 Web API**
- **React frontend**
- **MS SQL Server**
- Property inspection / handover workflow
- Inspection images
- Defects / activities / workspaces / items

The important point is that **YOLO should not replace your .NET application**. It should become an **AI inspection service** that analyzes inspection photos and sends structured results back to your .NET API.





1. What YOLO can do for your property inspection

For example, an inspector takes a photo of a bathroom:

```
Bathroom Photo
      ↓
     YOLO
      ↓
 ┌──────────────────────┐
 │ Tile                 │
 │ Wall                 │
 │ Floor                │
 │ Sink                 │
 │ Toilet               │
 │ Crack                │
 │ Stain                │
 │ Broken Tile          │
 └──────────────────────┘
      ↓
Inspection Result
      ↓
```

```
.NET API
      ↓
MS SQL
```

You could eventually have YOLO identify things such as:

Property defects

Category	Possible detection
Walls	Crack, hole, stain, paint damage
Ceiling	Crack, water stain, damaged paint
Floor	Broken tile, cracked tile, stain
Doors	Damage, scratch, misalignment
Windows	Broken glass, damaged frame
Bathroom	Missing sealant, tile damage, leakage evidence
Kitchen	Cabinet damage, countertop damage
Electrical	Socket damage, switch damage
HVAC	Vent damage, missing cover
Fixtures	Sink damage, toilet damage, faucet damage

But there is an important distinction:

YOLO doesn't magically know your property defects.

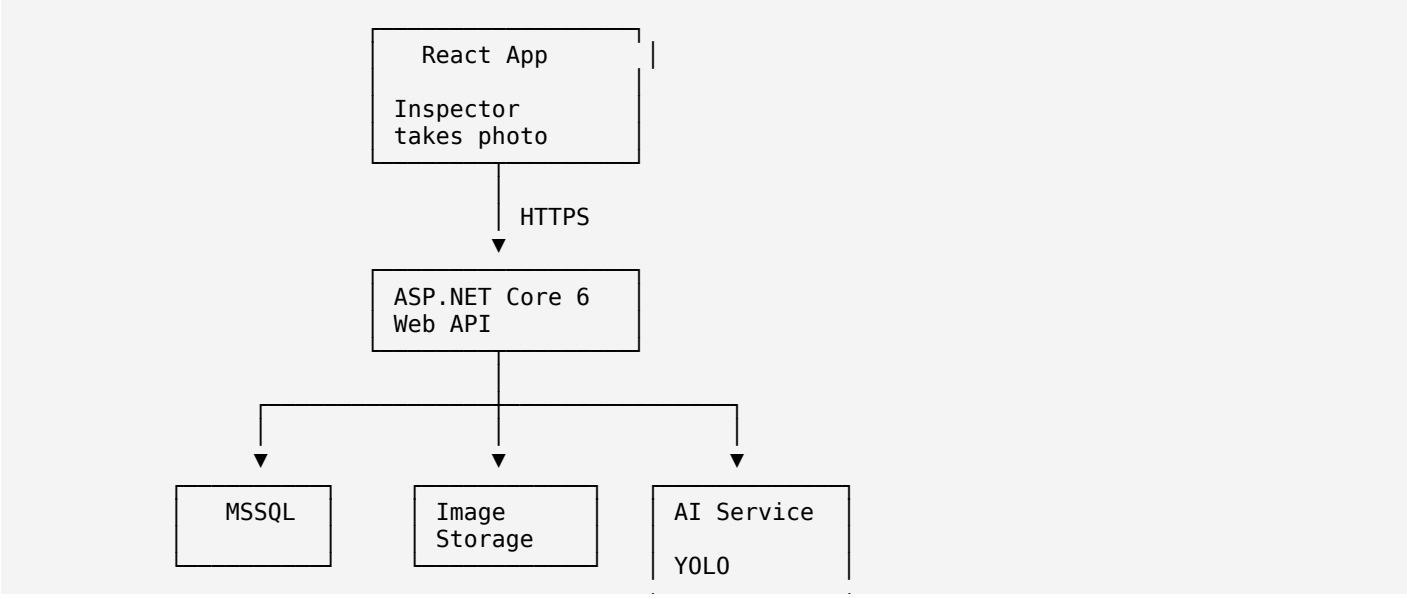
You need to train it using **your own inspection images and defect categories**.

Ultralytics supports custom training, validation, prediction and export to deployment formats such as ONNX. Ultralytics Docs+1

2. I would design your architecture like this

Don't put the YOLO Python code directly inside your ASP.NET API.

Instead:



```
                    └──────────────┘
                           │
                           ▼
                  Detection Results
                           │
                           ▼
                     ASP.NET Core
                           │
                           ▼
                        MSSQL
```

I recommend this architecture because your business workflow stays in .NET.

---

3. Your current application can remain almost unchanged

You probably have something conceptually like:

```
Property
   ↓
Unit
   ↓
Inspection
   ↓
Workspace
   ↓
Activity
   ↓
Activity Item
   ↓
Defect
   ↓
Images
```

We can add an AI layer:

```
Inspection
   │
   ├── Manual Inspection
   │
   └── AI Inspection
           │
           ├── Image
           ├── YOLO Detection
           ├── Defect
           ├── Confidence
           ├── Bounding Box
           └── AI Recommendation
```

This is much better than creating a completely separate AI application.

---

4. Example real workflow

Suppose the inspector opens:

```
Apartment A-1205
   ↓
Bathroom
   ↓
Floor
```

React displays:

```
┌──────────────────────────────────────┐
│ Bathroom > Floor                     │
│                                      │
│        [ Camera ]                    │
│                                      │
│      Take Inspection Photo           │
└──────────────────────────────────────┘
```

Inspector takes:

```
floor.jpg
```

React uploads:

```
http
POST /api/inspection/images
```

Your .NET API stores the image.

Then:

```
.NET API
   ↓
AI Inspection Service
   ↓
YOLO
```

YOLO returns:

```
JSON
{
  "detections": [
    {
      "class": "broken_tile",
      "confidence": 0.94,
      "x": 412,
      "y": 265,
      "width": 180,
      "height": 130
    }
  ]
}
```

Your .NET application converts that into a business result:

```
JSON
{
  "inspectionId": 12542,
  "workspace": "Bathroom",
  "activity": "Floor",
  "defect": "Broken Tile",
  "confidence": 0.94,
  "status": "AI_DETECTED"
}
```

Then the inspector sees:

```
┌──────────────────────────────────────┐
│          PROPERTY IMAGE              │
│                                      │
```

```
┌──────────────────────────────────────┐
│        ┌──────────────┐              │
│        │ BROKEN TILE  │              │
│        │    94%       │              │
│        └──────────────┘              │
│                                      │
└──────────────────────────────────────┘

AI detected:

⚠ Broken Tile
Confidence: 94%

[ Accept ]   [ Reject ]   [ Edit ]
```

This human confirmation step is extremely important.

Don't automatically create a final defect just because YOLO detected something.

---

5. What should YOLO actually detect?

I would **not start with 50–100 classes**.

Start with perhaps 10–15 high-value classes.

For example:

```
PropertyDefect
│
├── wall_crack
├── wall_hole
├── paint_damage
├── water_stain
├── floor_crack
├── broken_tile
├── tile_damage
├── door_damage
├── window_damage
├── glass_damage
├── ceiling_damage
├── socket_damage
├── fixture_damage
└── leakage_sign
```

Then expand.

---

6. Detection vs segmentation

This is particularly important for your project.

YOLO supports both **object detection** and **instance segmentation**. Ultralytics Docs+1

Detection

Returns:

```
┌──────────────┐
│    Crack     │
└──────────────┘
```

This is a bounding box.

Good for:

- Broken tile
- Door
- Socket
- Window
- Fixture

Segmentation

Instead of:

you get the **actual shape of the defect**.

For property inspection, segmentation can become very useful for:

- cracks
- water damage
- paint damage
- stains
- damaged areas

So eventually I'd use:

```
YOLO Detection
+
YOLO Segmentation
```

rather than detection only.

---

7. The biggest part is your dataset

This is where your existing application becomes extremely valuable.

You already have inspection history.

Suppose your database contains:

```
InspectionID
PropertyID
WorkspaceID
ActivityID
DefectID
ImageID
```

```
DefectType
```

and you have:

```
100,000 inspection images
```

You can create a training dataset.

For example:

```
AI Dataset

images/
   bathroom_001.jpg
   bathroom_002.jpg
   wall_001.jpg
   floor_001.jpg

labels/
   bathroom_001.txt
   bathroom_002.txt
   wall_001.txt
   floor_001.txt
```

Your historical inspection data becomes your **AI training asset**.

---

8. But don't train from MS SQL directly

This is an important architectural distinction.

Your database contains:

```
DefectType = Broken Tile
```

but YOLO needs something like:

```
Image:
bathroom_001.jpg

Bounding box:
x1
y1
x2
y2

Class:
broken_tile
```

Therefore you need an **annotation pipeline**.

Something like:

```
MSSQL
   ↓
Historical inspection records
   ↓
Image storage
   ↓
Annotation tool
   ↓
YOLO Dataset
   ↓
```

```
Training
   ↓
best.pt
   ↓
ONNX
```

9. Training architecture

I would keep training completely separate from production.

```
                TRAINING ENVIRONMENT

Inspection Images
        ↓
Annotation
        ↓
YOLO Dataset
        ↓
GPU Machine
        ↓
YOLO Training
        ↓
best.pt
        ↓
Validation
        ↓
ONNX
        ↓
Model Registry
```

Production:

```
React
  ↓
.NET API
  ↓
AI Service
  ↓
best.onnx
  ↓
Detection
  ↓
.NET
  ↓
MSSQL
```

Ultralytics supports exporting trained YOLO models to ONNX, which makes this architecture practical.
Ultralytics Docs+1

---

10. You have two choices for the AI service

Option A — Python AI service

This is what I recommend initially.

```
ASP.NET Core
        ↓
HTTP
        ↓
Python FastAPI
        ↓
```

```
YOLO
```

For example:

```
POST /predict
```

Request:

```
image.jpg
```

Response:

```JSON
{
  "modelVersion": "property-defect-v1",
  "detections": [
    {
      "class": "broken_tile",
      "confidence": 0.94,
      "bbox": {
        "x": 412,
        "y": 265,
        "width": 180,
        "height": 130
      }
    }
  ]
}
```

This is the easiest way to get your first version working.

---

11. Option B — Run YOLO directly from .NET

This is also possible.

Train YOLO:

```
YOLO
 ↓
best.pt
 ↓
best.onnx
```

Then your .NET API can load:

```
best.onnx
```

using ONNX Runtime.

ONNX Runtime provides a C# API and supports CPU and GPU execution providers. ONNX Runtime

Architecture:

```
ASP.NET Core 6
       │
       ├── Inspection API
       │
       ├── Image Service
       │
```

```
        └── YOLO Inference
                └── ONNX Runtime
                        └── property-defect.onnx
```

This eliminates the Python inference service.

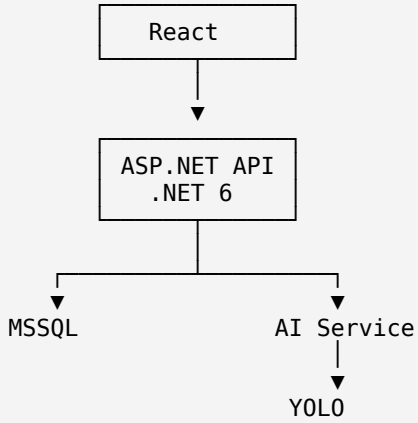
---

12. Which one would I choose for your application?

For your existing stack:

```
React
ASP.NET Core 6
MSSQL
```

I'd initially use:



Then, once the model is stable:

```
.NET API
   ↓
ONNX Runtime
   ↓
YOLO
```

if you want to simplify deployment.

---

13. Your database should also store AI information

I'd introduce something like:

InspectionImage

```
InspectionImage
------------------------
InspectionImageID
InspectionID
FilePath
CapturedDate
CreatedBy
```

Then:

AIInspectionResult

```
AIInspectionResult
------------------------
AIInspectionResultID
InspectionImageID
ModelVersion
ProcessedDate
ProcessingStatus
OverallConfidence
```

Then:

AIDetection

```
AIDetection
------------------------
AIDetectionID
AIInspectionResultID
ClassName
Confidence
X
Y
Width
Height
MaskData
IsAccepted
ReviewedBy
ReviewedDate
```

Then connect it to your existing defect table:

```
AIDetection
     │
     │ accepted
     ▼
HandOverDefect
```

This means AI doesn't destroy your existing workflow.

---

14. Very important: Model versioning

You should store:

```
ModelVersion
```

For example:

```
property-defect-v1
property-defect-v2
property-defect-v3
```

Why?

Suppose six months later:

```
v1 → 82% accuracy
```

```
v2 → 91% accuracy
```

You need to know:

> Which model generated this defect?

So:

```
AIInspectionResult
```

should contain:

```
ModelID
ModelVersion
Confidence
ProcessedDate
```

15. Confidence should affect workflow

Don't treat:

```
confidence = 0.51
```

the same as:

```
confidence = 0.97
```

I'd initially use something like:

```
Confidence >= 90%
        ↓
High confidence
        ↓
Auto-suggest defect
```

```
70% - 90%
        ↓
Medium confidence
        ↓
Inspector must review
```

```
< 70%
        ↓
Low confidence
        ↓
Don't suggest automatically
```

These thresholds should be determined from validation results rather than assumed permanently.

---

16. React UI becomes much more powerful

Instead of:

```
Take Photo
```

```
    ↓
Select Defect
    ↓
Write Comment
```

you can eventually have:

```
Take Photo
    ↓
AI Analysis
    ↓
┌──────────────────────────────┐
│ AI found 3 potential defects │
│                              │
│ 🔴 Broken Tile       94%     │
│ 🟠 Wall Crack        82%     │
│ 🟡 Paint Damage      71%     │
│                              │
│ [Review AI Findings]         │
└──────────────────────────────┘
```

Inspector can:

```
Accept
Reject
Edit
Add Manual Defect
```

This is a much better enterprise workflow.

---

17. Eventually you can build AI inspection scoring

This is where your project becomes much more interesting.

Suppose an apartment has:

```
Walls       → 3 defects
Floor       → 2 defects
Bathroom    → 4 defects
Kitchen     → 1 defect
Doors       → 0 defects
```

Your .NET system can calculate:

```
Inspection Score = 82/100
```

Then:

```
Apartment
    │
    ├── Bathroom       72%
    ├── Kitchen        91%
    ├── Living Room    87%
    ├── Bedroom        95%
    └── Balcony        89%
```

Now you have **property quality analytics**, not just image detection.

---

18. You can combine YOLO with your existing database

This is especially powerful for your application.

YOLO answers:

> **What is visible in the image?**

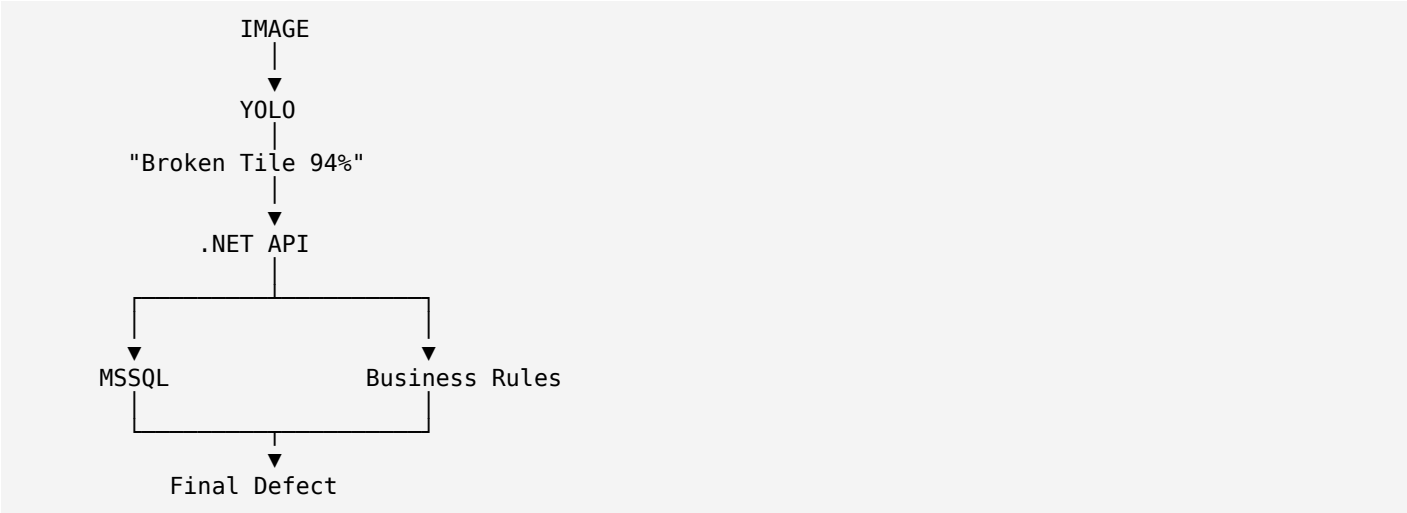
Your SQL database answers:

> **What property/workspace/activity does this image belong to?**

Your business rules answer:

> **Is this actually a defect according to our inspection workflow?**

So:



This separation is very important.

---

19. You can then add an LLM later

Since you're already exploring LLM/RAG/Qdrant, I wouldn't use an LLM for the actual visual detection.

Use:

```
YOLO
 ↓
Visual detection
```

and:

```
LLM
 ↓
Reasoning / explanation / summarization
```

For example:

YOLO:

```JSON
{
  "defect": "wall_crack",
  "confidence": 0.91
}
```

Database:

```
Property: A-1205
Room: Master Bedroom
Wall: North Wall
```

LLM can produce:

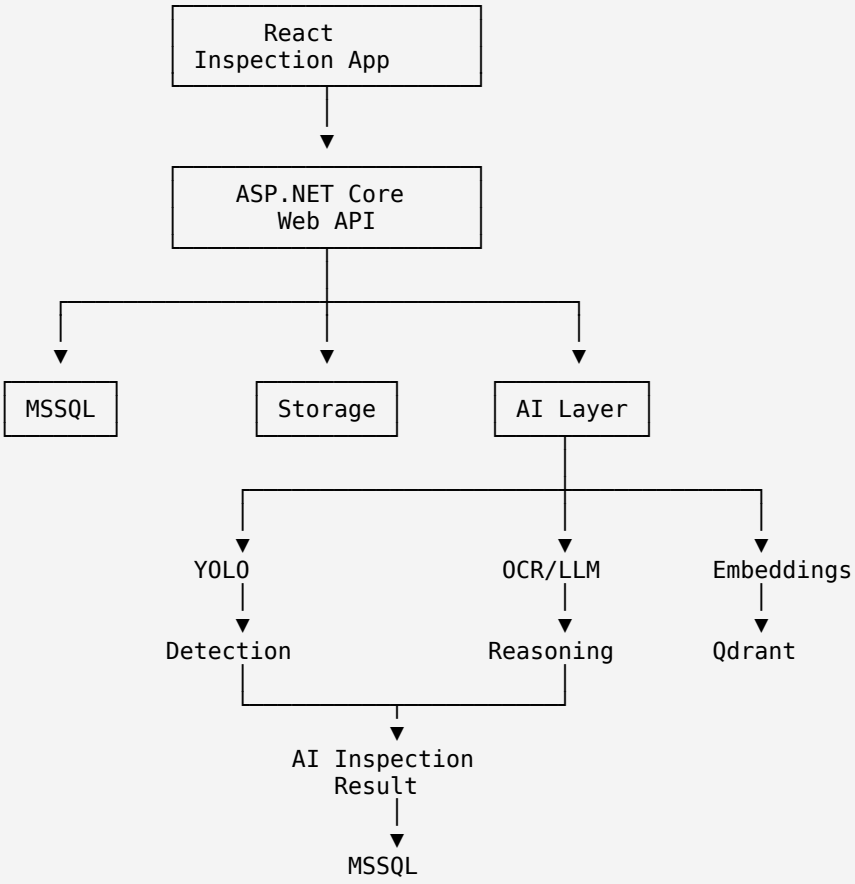
> A potential crack was detected on the north wall of the master bedroom. The inspector should verify whether the crack is structural or cosmetic before finalizing the defect.

That's much more appropriate.

---

20. Your complete future architecture

I'd ultimately target:



21. I would implement this in phases

Don't try to build everything at once.

Phase 1 — Proof of Concept

Use 5 defect classes:

```
crack
broken_tile
paint_damage
water_stain
door_damage
```

Take ~500–2,000 representative images and annotate them.

Train YOLO.

Goal:

```
Image
 ↓
YOLO
 ↓
Detection
```

Phase 2 — AI Service

Create:

```
AIInspectionService
```

with:

```
http
POST /api/ai/inspection/analyze
```

Input:

```
Image
InspectionID
WorkspaceID
ActivityID
```

Output:

```
JSON
{
  "success": true,
  "detections": [
    {
      "defectType": "broken_tile",
      "confidence": 0.94,
      "boundingBox": {
        "x": 412,
        "y": 265,
        "width": 180,
        "height": 130
      }
    }
  ]
}
```

Phase 3 — Integrate with your existing defect workflow

```
YOLO Detection
       ↓
AIInspectionResult
       ↓
Inspector Review
       ↓
Accept
       ↓
Existing HandOverDefect
```

Phase 4 — Segmentation

Add:

```
Crack segmentation
Water damage segmentation
Paint damage segmentation
```

YOLO's segmentation models can provide pixel-level masks rather than only bounding boxes.
Ultralytics Docs

Phase 5 — AI Inspection Assistant

Then combine:

```
YOLO
+
MSSQL
+
LLM
+
Qdrant
```

to create:

```
Property Inspection AI Assistant
```

Example:

> **AI Inspection Summary**
>
> 7 potential defects were detected.
>
> 3 high-confidence defects:
>
> - Broken tile — 94%
> - Wall crack — 92%
> - Paint damage — 91%
>
> 2 medium-confidence defects require inspector verification.
>
> Similar defects were previously reported in this workspace in 14 inspections.

That's where your existing **MS SQL + Qdrant + LLM work** becomes very valuable.

My recommendation for your specific application

Don't start by trying to detect every possible property defect.

Start with this architecture:

```
React
  ↓
.NET 6 API
  ↓
Image Storage
  ↓
YOLO AI Service
  ↓
Detection
  ↓
.NET Business Rules
  ↓
AIInspectionResult
  ↓
Inspector Confirmation
  ↓
Existing Defect Workflow
  ↓
MSSQL
```

And **train YOLO using your real historical property inspection images**, not generic COCO images. That will give you a system tailored to your actual handover/inspection domain.

Ultralytics currently supports custom YOLO training and exporting models to ONNX, while ONNX Runtime provides a C# inference API, so you have a clean path from experimentation to a .NET-integrated production deployment. Ultralytics Docs+2

The next step I recommend

We can implement this **step-by-step against your actual Property Inspection application**, starting with **Phase 1: design the SQL tables + .NET API contracts + YOLO dataset structure**, then build the Python YOLO service, train a first defect model, and finally integrate it into your React inspection screen.